

Arrays and performance in Java

Algorithms and data structures ID1021

Johan Montelius

Fall 2024

Introduction

In this assignment you should explore the efficiency of different operations over an array of elements. We take for granted that you have been working with arrays before so this should not be a problem, focus will be on performance measurement, presenting numbers and realizing that there is a fundamental difference between what is called $O(1)$, $O(n)$ and $O(n^2)$ (more on this later).

You should benchmark three different operations and determine how the execution time differs with the size of the array. The three operations are:

- Random access : reading or writing a value at a random location in an array.
- Search : searching through an array looking for an item.
- Duplicates : finding all common values in two arrays.

The implementation of these operations are quite straight forward, this is not problem. The problem is to do the benchmark, present the results and describe the general behavior for these operations.

Random access

When trying to measure how long time an operation takes, one is limited by the resolution of the clock. If we have a clock that measures microseconds then obviously it will be very hard to measure an operation that only takes a few nanoseconds.

the clock

In order to measure the performance we need a method to measure time. Your first task is to figure out the accuracy of this clock. Run the following code and present your conclusion in the report.

```
for (int i = 0; i < 10; i++) {
    long n0 = System.nanoTime();
    long n1 = System.nanoTime();
    System.out.println(" resolution " + (n1 - n0) + " nanoseconds");
}
```

Will the clock be accurate enough to measure how long time it takes to perform a single array access? Let's try the following:

```
int[] given = {0,1,2,3,4,5,6,7,8,9};
int sum = 0;

for (int i = 0; i < 10; i++) {
    long t0 = System.nanoTime();
    sum += given[i];
    long t1 = System.nanoTime();
    System.out.println("one operation in " + (t1 - t0) + " ns");
}
```

As you probably notice the time to access a value in an array is very small in comparison to the resolution of the clock. In order to get a better understanding of the time it takes to perform an access operation we would have to do a couple of hundred and measure the time it takes to do them all.

We preferably also want to make the access random (at random locations in the array) to prevent caching from playing a role (if we read from the same location or consecutive locations then the value will definitely be in the cache and give us a false impression). To do random read operations we could use a library procedure but we need to understand what we measure.

```
int[] given = {0,1,2,3,4,5,6,7,8,9};

Random rnd = new Random();

int sum = 0;
long t0 = System.nanoTime();
for (int i = 0; i < 1000; i++) {
    sum += given[rnd.nextInt(10)];
}
```

```

long t1 = System.nanoTime();
System.out.println(" one read operation in " + (t1 - t0)/1000 + " nanoseconds");

```

What is it that we now are measuring? The time it takes to create a random number is much longer than accessing an element in an array so we are measuring the wrong thing. In order to measure the time it takes to do a number of random access operation in an array of size n we could do some thing like this:

```

public static void int run(int[] array, int[] indx) {
    int sum = 0;
    for (int i = 0; i < indx.length() ; i++) {
        sum = sum + arr[indx[i]];
    }
    return sum;
}

public static long bench(int n, int loop) {

    int[] array = new int[n];
    for (int i = 0; i < n; i++) array[i] = i;

    Random rnd = new Random();

    int[] indx = new int[loop];
    for (int i = 0; i < loop; i++)  indx[i] = rnd.nextInt(n);

    int sum = 0;
    long t0 = System.nanoTime();
    run(array, indx);
    long t1 = System.nanoTime();
    return (t1 - t0);
}

```

In this program we first create the `array`, of size n , that we are to access but then also create an array `indx`, of size `loop`, of random indices that we are going to use to access the array. The actual loop now first selects the random index from the `indx` array and then use the index to access the target `array`.

If we are now picky we are measuring more than one array read operation. In each loop we: compare i to `loop`, access the array `indx`, access the array `array`, increment `sum` and increment i . You can remove the two read operations and simply increment the sum by one to see how much time the loop overhead is. As you will probably realize we're measuring the loop more than the read operations.

We ignore this for now since our goal is to determine how the access time differs when we change the size of the array. In the future we will measure the time to perform more complex operations that will take very much longer time so the time to do the loop will be negligible.

Try the following benchmark a couple of times.

```
public static void main(String[] arg) {

    for (int i = 0; i < 10; i++) {
        long t = bench(1000, 1000);
        System.out.println(" access: " + t + " ns");
    }
}
```

As you see, the measurements we get vary and the question is how we should report this. You could of course report the average or median of a hundred measurements ...or you could report the minimum value. Let's try this to see what we are talking about:

```
public static void main(String[] arg) {

    int n = 1000;
    int loop = 1000;
    int k = 10;

    long min = Long.MAX_VALUE;
    long max = 0;
    long total = 0;

    for (int i = 0; i < k; i++) {
        long t = bench(n, loop);
        if (t > max) max = t;
        if (t < min) min = t;
        total += t;
    }

    System.out.println(" avg: " + ((double) total)/loop/k);
    System.out.println(" min: " + ((double) min)/loop);
    System.out.println(" max: " + ((double) max)/loop);
}
```

If you run this a couple of times you will probably see that the maximum time varies quite a lot. This is understandable since you're running on top of an operating system that has a lot of things to do and once in a while it decides to pause our program. The minimum time is, in comparison, almost

carved in stone; there is an exact minimum value and the more times we try you will see that we more often hit this value.

The average value might be of interest but since it is very influenced by the worst case number it only adds noise in what we want to show. The median is better for describing the behavior but why not go for the most predictable one - the minimum value. If the minimum value is all that we are after we can of course simplify the program.

There is one other thing that we have to take care of when we are benchmarking programs in Java and that is the so called "just-in-time" compiler or JIT for short. When we compile a program in Java it is turned into so called *byte code*. This is a good enough compilation with many advantages but it is not the most efficient. During execution, if the runtime system detects that a method is used extensively it will again compile it into *machine code*. The improvement can be quite dramatic but there is no explicit way of controlling when and if this extra compilation phase kicks in.

One way to solve this is to do a run of the method that we are benchmarking before we actually do the benchmarking. We then make sure that the method is executed for sufficient time to make the runtime system realize that it should let the JIT-compiler do its job.

Try your benchmark with this small change and see if it makes a difference:

```
        :
        bench(n, 1000000);

        for (int i = 0; i < k; i++) {
            long t = bench(n, loop);
            if (t > max) max = t;
            if (t < min) min = t;
            total += t;
        }
        :
    }
```

Now we only add some code where we get the minimum time of the different sizes of an array.

```
public static void main(String[] arg) {

    int[] sizes = {100, 200, 400, 800, 1600, 3200};

    // JIT warmup
    bench(1000, 10000000);
}
```

```

int loop = 1000;
int k = 10;

for(int n : sizes) {
    long min = Long.MAX_VALUE;
    for (int i = 0; i < k; i++) {
        long t = bench(n, loop);
        if (t < min) min = t;
    }
    System.out.println(n + " " + ((double) min)/loop + " ns");
}
}

```

The reason why you should go through these steps is not because I want you to memorize the one and only way of measuring performance. The reason is that I want you to realize that measuring something so trivial as the time it takes to access an element is quite difficult.

In the future we will measure things that are a bit easier to measure, mostly because they take longer time, but you should always question your method and make user that you measure what you think you're measuring.

presenting execution time

When you report execution times it is quite ridiculous to report something like 123456789 *ns* - even if this is actually the result that you got. The reason that it is ridiculous is that the execution time will most likely change very much if you run the benchmark again. If the next run results in 124356789 *ns* then it is rather pointless to present the runtime using nine significant figures.

Your choice of the number of significant figures should reflect how stable your measurements are. If you run your benchmarks on a regular laptop there is a lot of things going on in the background that will effect the execution time. My guess is that you will not be able measure anything with more than three figures of accuracy and in most cases two will do fine.

Even if you can make a stable measurement with three significant figures it might not be how you should report it. If you want to show that a sequence of measurements increase with a factor 2 for each measurement, large numbers will only be in the way.

Take a look at the table 1. The conclusion might be correct but it is not "clearly seen" at all.

Always think about the message that you want to deliver and then use as few numbers as possible to convey that message. If the doubling of execution time was the message, then Table 2 might be a better idea.

n	100	200
prgm1	12345678 μs	22345678 μs
prgm2	14325678 μs	56213478 μs

Table 1: As clearly seen - prgm1 double the execution time whereas prgm2 almost increase with a factor four.

n	100	200
prgm1	12 <i>ms</i>	22 <i>ms</i>
prgm2	14 <i>ms</i>	56 <i>ms</i>

Table 2: Much better

When you present numbers also think about readability. To say 23000000 μs is of course only using two significant figures but is very hard to compare this measurement with 1200000 μs ... is that half the execution time or ...? Also refrain from using mathematical notations such as $1.23 \times 10^7 ns$, especially when you want someone to quickly see the relationship with $2.4 \times 10^6 ns$, the information is of course all there but you don't make the connection without thinking - how about 12 *ms* compared to 2.4 *ms*.

Search for an item

Once we know how to set up a nice benchmark we can explore a simple search algorithm and see how the execution time varies with the size of the array.

When we setup the benchmark we want to capture the estimated time it would take to search for a random key in an array of unsorted keys. We assume that the number of keys is larger the size of the array - if we only had the keys 1, 2 and 3 we would probably find one very quickly even if the array was very large. So when we search for a given key it might be that we search through the whole array without finding it.

If we follow the pattern in the previous section we could try the following:

```
private static long search(int n, int loop) {

    Random rnd = new Random();

    int[] array = new int[n];
    for (int i = 0; i < n; i++) {
        array[i] = rnd.nextInt(n*2);
    }
}
```

```

int[] keys = new int[n];
for (int k = 0; k < loop; k++) {
    keys[k] = rnd.nextInt(n*2);
}

int sum = 0;

long t0 = System.nanoTime();
for (int i = 0; i < loop; i++) {
    int key = keys[i];
    for (int j = 0; j < n; j++) {
        if (key == array[j]) {
            sum++;
            break;
        }
    }
}
long t1 = System.nanoTime();
return (t1 - t0);
}

```

How would you present the figures you get and how would you explain the result? Can you find a simple polynomial that roughly describes the execution time?

Search for duplicates

So now for the final task, finding duplicated numbers in two arrays of size n i.e. an element in the first array that also is present in the second array (we can assume that the arrays themselves do not contain duplicates and if they do the answer might be undefined).

This task is very similar to the search exercise and we will use the same strategy; for each key in the first array try to find it in the second array. The only difference is that now both arrays grow in size; a small change it might seem but it makes a huge difference. If the size of the array is large enough (> 100 elements) we could skip the loop parameter; the time to find the duplicates will be long enough to hide the uncertainty of the clock.

```

private static long duplicates(int n) {

    Random rnd = new Random();

    int[] array_a = new int[n];
    for (int i = 0; i < n; i++) {

```



```

        array_a[i] = rnd.nextInt(n*2);
    }

    int[] array_b = new int[n];
    for (int i = 0; i < n; i++) {
        array_b[i] = rnd.nextInt(n*2);
    }

    int sum = 0;

    long t0 = System.nanoTime();

    for (int k = 0; k < loop; k++) {
        for (int i = 0; i < n; i++) {
            int key = array_a[i];
            for (int j = 0; j < n; j++) {
                if (key == array_b[j]) {
                    sum++;
                    break;
                }
            }
        }
    }
    long t1 = System.nanoTime();
    return t1 - t0;
}

```

How would you present the figures you get and how would you explain the result? Can you find a simple polynomial that roughly describes the execution time for finding duplicates in two arrays of size n ?